WEJ LAB

Explainable AI: Variable Importance, SHAP, and LIME

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-11-11

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

- Explain why we need to explain "black-box" models
- Distinguish global from local, and model-specific from model-agnostic explanations
- Compute and interpret permutation importance
- Compute SHAP values for a tree model and read summary, dependence, and individual plots
- Explain a single prediction with LIME, and know the limits of both

WEJ LAB

Why explain a model?

- **Trust:** clinicians won't act on a score they can't question
- **Debugging:** is the model using signal, or a **shortcut**?
- **Discovery:** which genes or risk factors drive the prediction?
- **Accountability:** regulators and patients increasingly expect reasons

Shortcut example: pneumonia models trained on chest X-rays partly learned *which hospital* an image came from (Zech et al., *PLOS Medicine* 2018).

RUTGERS HEALTH
Center for Data Science

Kinds of explanations

- **Intrinsically interpretable** models: linear/logistic regression, a small tree
- **Post hoc** explanations of a black box (forest, boosting, neural net):
  - **Global:** how does the model behave overall? (importance, dependence)
  - **Local:** why did *this* patient get *this* prediction? (SHAP, LIME)
- **Model-specific** (uses the model's internals)
  vs. **model-agnostic** (only needs predictions)

Running example: diabetes in NHANES

Back to the NHANES adults from the regularization lecture: predict **diabetes** from age, BMI, blood pressure, cholesterol, and lifestyle. Ordinal factors are coded as numbers (e.g., `HealthGen`: 1 = Excellent, . . . , 5 = Poor).

This time, with **XGBoost**: accurate, but not interpretable on its own.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Running example: the data

```r
d <- NHANES %>% filter(Age >= 20) %>% distinct(ID, .keep_all = TRUE) %>%
  select(Diabetes, Age, Gender, Education, Poverty, BMI, Weight, Height,
         Pulse, BPSysAve, BPDiaAve, TotChol, DirectChol, SleepHrsNight,
         PhysActive, Smoke100, HealthGen) %>%
  drop_na() %>%
  mutate(across(c(Education, HealthGen), as.numeric),    # ordinal -> 1, 2, ...
         Male = as.numeric(Gender == "male"),
         PhysActive = as.numeric(PhysActive == "Yes"),
         Smoke100 = as.numeric(Smoke100 == "Yes")) %>%
  select(-Gender)
X <- as.matrix(select(d, -Diabetes))
y <- as.numeric(d$Diabetes == "Yes")
dim(X); mean(y)
```

```
## [1] 3570   16
```

```
## [1] 0.1173669
```

Running example: XGBoost model

Train/test split; number of rounds by 5-fold CV with early stopping:

```
set.seed(1)
idx <- createDataPartition(d$Diabetes, p = 0.7, list = FALSE)
X_train <- X[idx, ];  y_train <- y[idx]
X_test  <- X[-idx, ]; y_test  <- y[-idx]
dtrain  <- xgb.DMatrix(X_train, label = y_train)
params  <- list(objective = "binary:logistic", eval_metric = "auc", eta = 0.05,
                max_depth = 3, subsample = 0.8, colsample_bytree = 0.8)
cv <- xgb.cv(params = params, data = dtrain, nrounds = 1000, nfold = 5,
             early_stopping_rounds = 30, verbose = 0)
best_n  <- which.max(cv$evaluation_log$test_auc_mean)
xgb_fit <- xgb.train(params = params, data = dtrain, nrounds = best_n)
p_test  <- predict(xgb_fit, X_test)
auc(y_test, p_test, quiet = TRUE)
```

```
## Area under the curve: 0.8617
```

WEJ LAB

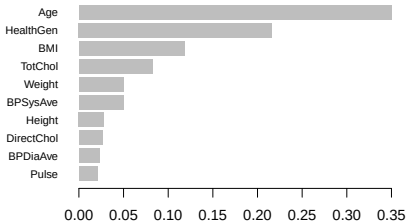
Global explanations

RUTGERS HEALTH
Center for Data Science

Built-in importance (model-specific)

XGBoost's **gain**: total improvement in the loss from splits on each feature:

```
imp <- xgb.importance(model = xgb_fit)
xgb.plot.importance(imp, top_n = 10)
```



Permutation importance (model-agnostic)

Idea: shuffle one feature in the **test set**, breaking its link to the outcome. How much worse does the model get?

- Big drop $\rightarrow$ the model relies on that feature
- Works for **any** model: only needs predictions
- Measured on held-out data, so it reflects **generalization**, not memorization

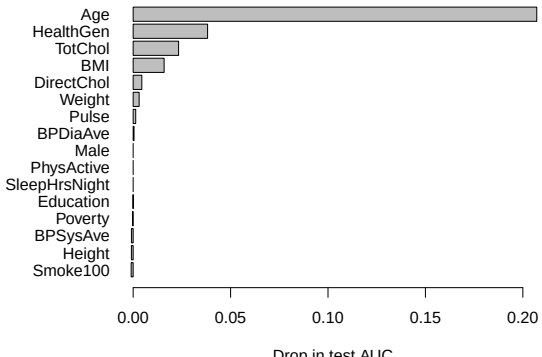
WEJ LAB
RUTGERS HEALTH
Center for Data Science

Permutation importance

```r
auc0 <- as.numeric(auc(y_test, p_test, quiet = TRUE))
set.seed(1)
perm_imp <- sapply(colnames(X_test), function(j){
  mean(replicate(10, {                          # average over 10 shuffles
    X_perm <- X_test
    X_perm[, j] <- sample(X_perm[, j])          # shuffle one column
    auc0 - as.numeric(auc(y_test, predict(xgb_fit, X_perm), quiet = TRUE))
  }))
})
barplot(sort(perm_imp), horiz = TRUE, las = 1, xlab = "Drop in test AUC")
```

Permutation importance

Age dominates, then self-rated health, cholesterol, and BMI. Weight gets a little credit too ($r = 0.89$ with BMI):



WEJ LAB

SHAP

RUTGERS HEALTH
Center for Data Science

SHAP: SHapley Additive exPlanations

From cooperative game theory (Shapley 1953; Lundberg & Lee 2017):

- The **features** are players on a team; the **prediction** is the payout
- A feature's **SHAP value** ϕ_j is its average marginal contribution, over all orders in which features could join the team
- Gives a **local** explanation for every individual

SHAP: additivity

For each individual, the SHAP values add up exactly to the prediction:

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{p} \phi_j(\mathbf{x})$$

- ϕ_0 = the average prediction (baseline)
- $\phi_j > 0$ pushes this person's prediction **up**; $\phi_j < 0$ pushes it **down**
- For XGBoost classification, the scale is **log-odds**

SHAP for trees: TreeSHAP

Exact Shapley values need all 2^p feature subsets. For tree ensembles, **TreeSHAP** computes them exactly and fast. XGBoost has it built in:

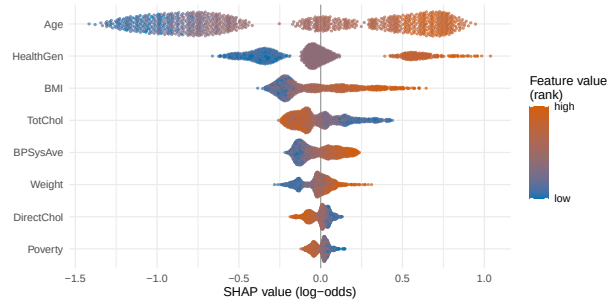
```
shap <- predict(xgb_fit, X_test, predcontrib = TRUE)   # one column per feature + BIAS
dim(shap)
```

```
## [1] 1070   17
```

```
phi0 <- shap[1, ncol(shap)]                  # baseline (log-odds)
shap <- shap[, -ncol(shap)]
# check additivity: baseline + sum of SHAP = predicted log-odds
all.equal(phi0 + rowSums(shap), qlogis(p_test), tolerance = 1e-4)
```

```
## [1] TRUE
```

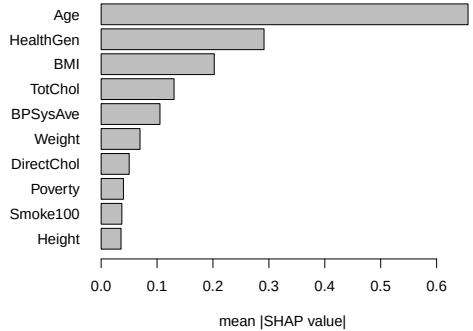
SHAP summary (beeswarm) plot



One dot per person per feature. Older age, poorer health, higher BMI (orange) push risk **up**. Higher cholesterol pushes it **down**: plausibly statin treatment in people with diabetes. **Not causal!**

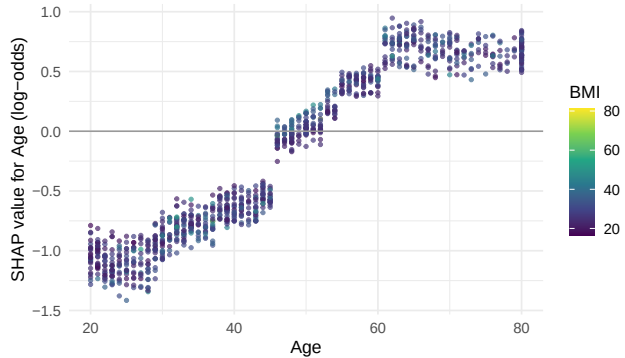
SHAP global importance

Average the **absolute** SHAP values over people for a global ranking:



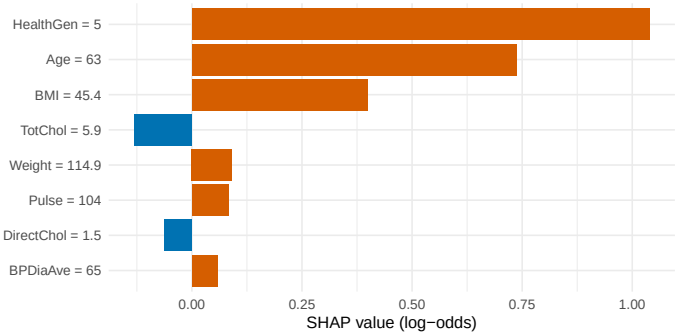
SHAP dependence plot

How the model uses **Age** (each dot is one person; color = BMI):



A **non-linear** effect: risk rises steeply after about 40–50, then flattens.

SHAP for one person



The highest-risk test patient: predicted probability 0.57 vs. an average of 0.12 (baseline ϕ_0).
Diabetes: yes.

SHAP in practice: packages

The plots above in a few lines with **shapviz** (works with `xgboost`, `ranger`, ...):

```
library(shapviz)
sv <- shapviz(xgb_fit, X_pred = X_test)
sv_importance(sv, kind = "beeswarm")    # summary plot
sv_dependence(sv, "Age")                # dependence plot
sv_waterfall(sv, row_id = 1)            # one person
```

For any `caret` model (slower, Monte Carlo approximation): `iml::Shapley`, or `kernelshap::kernelshap()` + `shapviz`.

WEJ LAB

LIME

RUTGERS HEALTH
Center for Data Science

LIME: Local Interpretable Model-agnostic Explanations

Idea (Ribeiro et al. 2016): near one individual **x**, a complex model is roughly **linear**.

1. **Perturb x**: generate many "neighbors"
2. **Predict** the neighbors with the black-box model
3. **Weight** neighbors by closeness to **x**
4. **Fit** a simple (sparse linear) model to the weighted neighbors: its coefficients are the explanation

WEJ LAB
RUTGERS HEALTH
Center for Data Science

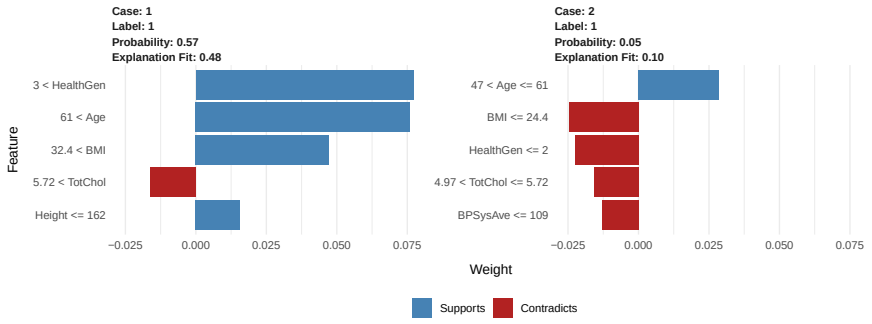
LIME in R

The **lime** package works with caret, xgboost, and other models:

```
explainer <- lime(as.data.frame(X_train), xgb_fit)  # learns feature distributions
cases <- c(which.max(p_test), which.min(abs(p_test - 0.05)))  # high and low risk
set.seed(1)
lime_res <- explain(as.data.frame(X_test[cases, ]), explainer,
                    labels = "1",      # explain P(diabetes)
                    n_features = 5)
```

LIME in R

```
plot_features(lime_res)
```



LIME vs. SHAP

	LIME	SHAP
Asks	why did *this* prediction happen?	how much did each feature contribute?
Basis	a simple local surrogate model	game-theoretic credit assignment
Guarantees	none: depends on sampling and kernel	additive and consistent
Cost	fast	exact and fast for trees (TreeSHAP); slow otherwise
Scope	local only	local *and* global

LIME: a quick look at one case. SHAP: when the attribution must be defensible.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Cautions

- Explanations describe **the model**, not biology: **not causal** (cholesterol!)
- **Correlated features** (BMI, weight) share or swap credit
- **LIME is unstable**: rerun with another seed and the explanation can change
- A good explanation of a **bad model** is still a bad model: evaluate first
- Always sanity-check against **domain knowledge**

Looking ahead

In **Homework 4** you will use these tools on the **TB NanoString** data: which genes push a prediction toward TB, and which toward latent infection?

Summary

- Global explanations: built-in and **permutation** importance
- **SHAP**: additive, game-theoretic attributions; summary, dependence, and individual plots
- **TreeSHAP** makes SHAP exact and fast for tree ensembles (`predcontrib = TRUE`)
- **LIME**: a local linear surrogate around one prediction; fast but unstable
- Explanations are **predictive, not causal**; beware correlated features

Python companion

Python versions of these examples (`xgboost`, `scikit-learn`'s `permutation_importance`, and the `shap` package):

`python/week7/explainableAI.ipynb`

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] ggbeeswarm_0.7.2 lime_0.5.5       xgboost_3.2.1.1  NHANES_2.1.4
##  [5] pROC_1.18.5      nnet_7.3-19      neuralnet_1.44.2 lubridate_1.9.3
##  [9] forcats_1.0.0    stringr_1.5.1    dplyr_1.1.4      purrr_1.0.2
## [13] readr_2.1.5      tidyr_1.3.1      tibble_3.2.1     tidyverse_2.0.0
## [17] caret_6.0-94     lattice_0.22-5   ggplot2_3.4.4
##
## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.0     viridisLite_0.4.2    timeDate_4032.109
```